

光与朽游戏软件源程序

```

using UnityEngine;
using System.Collections.Generic;
namespace NeonGamblingTower.Laser
{
    /// <summary>
    /// 激光 LineRenderer 控制器 - Step 2 反射逻辑
    /// 功能：旋转控制、长度控制、宽度控制、VFX 同步、墙壁反射
    /// 使用世界空间坐标，支持最多 3 次反射
    /// </summary>
    public class LaserLineRendererController : MonoBehaviour
    {
        #region Inspector Fields
        [Header("=== 组件引用 ===")]
        [SerializeField] private Transform laserPivot;           // 旋转控制节点
        [SerializeField] private LineRenderer lineRenderer;      // 激光 LineRenderer
        [SerializeField] private Transform startVFX;             // 起点粒子特效
        [SerializeField] private Transform endVFX;               // 终点粒子特效
        [Header("=== VFX 子节点引用 ===")]
        [SerializeField] private Transform[] startVFXChildren;   // StartVFX 下需要缩放的子节点
        [SerializeField] private Transform[] endVFXChildren;     // EndVFX 下需要缩放的子节点
        [Header("=== 激光参数 ===")]
        [SerializeField] private float laserLength = 19f;        // 激光总长度
        [SerializeField] private float laserWidth = 0.5f;        // 激光宽度
        [SerializeField] private float startPointOffset = 0f;    // 起点 Y 轴偏移（本地坐标）
        [Header("=== 旋转控制 ===")]
        [SerializeField] private float rotationSpeed = 100f;     // 旋转速度
        [SerializeField] private float minAngle = -90f;          // 最小角度（右侧）
        [SerializeField] private float maxAngle = 90f;           // 最大角度（左侧）
        [Header("=== VFX 缩放 ===")]
        [SerializeField] private float baseWidth = 0.5f;         // 基础宽度（对应 VFX 缩放为 1）
        [SerializeField] private float baseVFXScale = 1f;        // 基础 VFX 缩放值
        [Header("=== 反射设置 ===")]
        [SerializeField] private int maxReflections = 0;          // 最大反射次数（0-3，由技能等级决定）
        [SerializeField] private LayerMask wallLayer;            // 墙壁 Layer
        [SerializeField] private LayerMask enemyLayer;           // 敌人 Layer（包含 Enemy 和 BouncingEnemy）
        [Header("=== 性能优化 ===")]
        [SerializeField] private float raycastInterval = 0.02f;  // Raycast 检测间隔（秒）
        [Header("=== 调试 ===")]
        [SerializeField] private bool showDebugGizmos = true;
        [SerializeField] private bool showDebugLogs = false;
        #endregion
        #region Private Variables
        // 旋转控制
        private float currentAngle = 0f;
        private Vector2 lastTouchPosition;
        private bool isDragging = false;
        // Raycast 优化
        private float lastRaycastTime;

```

```
// 激光路径点（世界坐标）
private List<Vector3> laserPoints = new List<Vector3>();
// 缓存
private Transform cachedTransform;
// 当前击中的目标信息
private bool hitEnemy = false;
private Vector3 hitPoint;
private int currentReflectionCount = 0;
#endregion
#region Unity Lifecycle
private void Awake()
{
    cachedTransform = transform;
}
private void Start()
{
    InitializeLaser();
}
private void Update()
{
    HandleRotationInput();
    // 定期执行 Raycast 检测
    if (Time.time - lastRaycastTime >= raycastInterval)
    {
        lastRaycastTime = Time.time;
        CalculateLaserPath();
    }
    UpdateLaserVisuals();
}
private void OnValidate()
{
    // 限制反射次数在 0-3 之间
    maxReflections = Mathf.Clamp(maxReflections, 0, 3);
    if (lineRenderer != null && Application.isPlaying)
    {
        CalculateLaserPath();
        UpdateLaserVisuals();
    }
}
#endregion
#region Initialization
/// <summary>
/// 初始化激光设置
/// </summary>
private void InitializeLaser()
{
    if (lineRenderer == null)
    {
        Debug.LogError("[LaserController] LineRenderer 引用为空!");
        return;
    }
}
```

```

}
// 确保使用世界空间
lineRenderer.useWorldSpace = true;
// 初始化路径点列表
laserPoints.Clear();
// 初始化 Layer（如果未设置）
if (wallLayer == 0)
{
    wallLayer = LayerMask.GetMask("Wall");
}
if (enemyLayer == 0)
{
    enemyLayer = LayerMask.GetMask("Enemy", "BouncingEnemy");
}
// 立即计算一次路径
CalculateLaserPath();
UpdateLaserVisuals();
Debug.Log($"[LaserController] 激光初始化完成 - 世界空间模式，最大反射次数：
{maxReflections}");
}
#endregion
#region Input Handling
/// <summary>
/// 处理旋转输入（支持触摸和鼠标）
/// </summary>
private void HandleRotationInput()
{
    // 触摸输入
    if (Input.touchCount > 0)
    {
        Touch touch = Input.GetTouch(0);
        HandleTouchInput(touch);
    }
    // 鼠标输入（编辑器测试用）
    else
    {
        HandleMouseInput();
    }
}
/// <summary>
/// 处理触摸输入
/// </summary>
private void HandleTouchInput(Touch touch)
{
    switch (touch.phase)
    {
        case TouchPhase.Began:
            lastTouchPosition = touch.position;
            isDragging = true;
            break;
    }
}

```

```

case TouchPhase.Moved:
if (isDragging)
{
float deltaX = touch.position.x - lastTouchPosition.x;
ApplyRotation(deltaX);
lastTouchPosition = touch.position;
}
break;
case TouchPhase.Ended:
case TouchPhase.Canceled:
isDragging = false;
break;
}
}
/// <summary>
/// 处理鼠标输入
/// </summary>
private void HandleMouseInput()
{
if (Input.GetMouseButtonDown(0))
{
lastTouchPosition = Input.mousePosition;
isDragging = true;
}
else if (Input.GetMouseButton(0) && isDragging)
{
float deltaX = Input.mousePosition.x - lastTouchPosition.x;
ApplyRotation(deltaX);
lastTouchPosition = Input.mousePosition;
}
else if (Input.GetMouseButtonUp(0))
{
isDragging = false;
}
}
/// <summary>
/// 应用旋转
/// </summary>
private void ApplyRotation(float deltaX)
{
float rotationDelta = -deltaX * rotationSpeed * Time.deltaTime;
currentAngle = Mathf.Clamp(currentAngle + rotationDelta, minAngle, maxAngle);
if (laserPivot != null)
{
laserPivot.localRotation = Quaternion.Euler(0, 0, currentAngle);
}
}
#endregion
#region Laser Path Calculation
/// <summary>

```

```
/// 计算激光路径（包含反射）
/// </summary>
private void CalculateLaserPath()
{
    laserPoints.Clear();
    hitEnemy = false;
    currentReflectionCount = 0;
    if (laserPivot == null) return;
    // 起点（世界坐标）
    Vector3 startPoint = CalculateStartPointWorld();
    laserPoints.Add(startPoint);
    // 初始方向（LaserPivot 的本地 Y 轴方向）
    Vector3 currentDirection = laserPivot.up;
    Vector3 currentPoint = startPoint;
    float remainingLength = laserLength;
    // 循环计算反射路径
    while (remainingLength > 0 && currentReflectionCount <= maxReflections)
    {
        // 执行射线检测
        RaycastResult result = PerformRaycast(currentPoint, currentDirection, remainingLength);
        if (result.hitSomething)
        {
            // 添加击中点
            laserPoints.Add(result.hitPoint);
            remainingLength -= result.hitDistance;
            if (result.hitEnemy)
            {
                // 击中敌人，停止
                hitEnemy = true;
                hitPoint = result.hitPoint;
                if (showDebugLogs)
                {
                    Debug.Log($"[LaserController] 击中敌人，激光停止 - 位置: {result.hitPoint}");
                }
                break;
            }
            else if (result.hitWall)
            {
                // 击中墙壁，检查是否可以反射
                if (currentReflectionCount < maxReflections && remainingLength > 0.1f)
                {
                    // 计算反射方向
                    currentDirection = Vector3.Reflect(currentDirection, result.hitNormal);
                    // 【修复】将起点沿反射方向偏移一小段距离，避免立即再次检测到同一面墙
                    currentPoint = result.hitPoint + currentDirection * 0.01f;
                    currentReflectionCount++;
                    if (showDebugLogs)
                    {
                        Debug.Log($"[LaserController] 反射 #{currentReflectionCount} - 位置: {result.hitPoint}, 新方向: {currentDirection}");
                    }
                }
            }
        }
    }
}
```

```

    }
    }
    else
    {
        // 达到最大反射次数或剩余长度不足，停止
        if (showDebugLogs)
        {
            Debug.Log($"[LaserController] 达到最大反射次数或剩余长度不足，停止");
        }
        break;
    }
    }
    }
    else
    {
        // 没有击中任何东西，延伸到最大长度
        Vector3 endPoint = currentPoint + currentDirection * remainingLength;
        laserPoints.Add(endPoint);
        if (showDebugLogs)
        {
            Debug.Log($"[LaserController] 激光延伸到最大长度 - 终点: {endPoint}");
        }
        break;
    }
    }
    // 确保至少有 2 个点
    if (laserPoints.Count < 2)
    {
        Vector3 endPoint = startPoint + laserPivot.up * laserLength;
        laserPoints.Add(endPoint);
    }
    }
    /// <summary>
    /// 执行射线检测
    /// </summary>
    private RaycastResult PerformRaycast(Vector3 origin, Vector3 direction, float maxDistance)
    {
        RaycastResult result = new RaycastResult();
        // 分别检测敌人和墙壁
        RaycastHit2D enemyHit = Physics2D.Raycast(origin, direction, maxDistance, enemyLayer);
        RaycastHit2D wallHit = Physics2D.Raycast(origin, direction, maxDistance, wallLayer);
        bool hasEnemyHit = enemyHit.collider != null;
        bool hasWallHit = wallHit.collider != null;
        // 根据优先级处理（优先敌人）
        if (hasEnemyHit && hasWallHit)
        {
            // 两者都击中，优先敌人（无论距离）
            result.hitSomething = true;
            result.hitEnemy = true;
            result.hitPoint = enemyHit.point;

```

```

result.hitNormal = enemyHit.normal;
result.hitDistance = enemyHit.distance;
result.hitCollider = enemyHit.collider;
}
else if (hasEnemyHit)
{
// 只击中敌人
result.hitSomething = true;
result.hitEnemy = true;
result.hitPoint = enemyHit.point;
result.hitNormal = enemyHit.normal;
result.hitDistance = enemyHit.distance;
result.hitCollider = enemyHit.collider;
}
else if (hasWallHit)
{
// 只击中墙壁
result.hitSomething = true;
result.hitWall = true;
result.hitPoint = wallHit.point;
result.hitNormal = wallHit.normal;
result.hitDistance = wallHit.distance;
result.hitCollider = wallHit.collider;
}
return result;
}
/// <summary>
/// 计算起点的世界坐标
/// </summary>
private Vector3 CalculateStartPointWorld()
{
if (laserPivot == null) return Vector3.zero;
Vector3 localStartPoint = new Vector3(0, startPointOffset, 0);
return laserPivot.TransformPoint(localStartPoint);
}
#endregion
#region Laser Visuals Update
/// <summary>
/// 更新激光视觉效果
/// </summary>
private void UpdateLaserVisuals()
{
if (lineRenderer == null || laserPoints.Count < 2) return;
// 更新 LineRenderer 点数
lineRenderer.positionCount = laserPoints.Count;
// 设置所有点的位置
for (int i = 0; i < laserPoints.Count; i++)
{
lineRenderer.SetPosition(i, laserPoints[i]);
}
}

```

```
// 更新宽度
UpdateLaserWidth();
// 更新 VFX 位置和缩放
UpdateVFXPositions();
UpdateVFXScale();
}
/// <summary>
/// 更新激光宽度
/// </summary>
private void UpdateLaserWidth()
{
    if (lineRenderer == null) return;
    lineRenderer.startWidth = laserWidth;
    lineRenderer.endWidth = laserWidth;
}
/// <summary>
/// 更新 VFX 位置
/// </summary>
private void UpdateVFXPositions()
{
    if (laserPoints.Count < 2) return;
    // StartVFX 在激光起点
    if (startVFX != null)
    {
        startVFX.position = laserPoints[0];
    }
    // EndVFX 在激光终点（最后一个点）
    if (endVFX != null)
    {
        endVFX.position = laserPoints[laserPoints.Count - 1];
    }
}
/// <summary>
/// 更新 VFX 子节点缩放（根据激光宽度）
/// </summary>
private void UpdateVFXScale()
{
    float widthRatio = laserWidth / baseWidth;
    float targetScale = baseVFXScale * widthRatio;
    // 缩放 StartVFX 的子节点
    if (startVFXChildren != null)
    {
        foreach (var child in startVFXChildren)
        {
            if (child != null)
            {
                child.localScale = Vector3.one * targetScale;
            }
        }
    }
}
```

```

// 缩放 EndVFX 的子节点
if (endVFXChildren != null)
{
    foreach (var child in endVFXChildren)
    {
        if (child != null)
        {
            child.localScale = Vector3.one * targetScale;
        }
    }
}
}
}
}
#endregion
#region Public API
/// <summary>
/// 设置激光长度
/// </summary>
public void SetLength(float length)
{
    laserLength = Mathf.Max(0, length);
    CalculateLaserPath();
    UpdateLaserVisuals();
}
/// <summary>
/// 设置激光宽度
/// </summary>
public void SetWidth(float width)
{
    laserWidth = Mathf.Max(0.01f, width);
    UpdateLaserVisuals();
}
/// <summary>
/// 设置旋转角度
/// </summary>
public void SetRotation(float angle)
{
    currentAngle = Mathf.Clamp(angle, minAngle, maxAngle);
    if (laserPivot != null)
    {
        laserPivot.localRotation = Quaternion.Euler(0, 0, currentAngle);
    }
    CalculateLaserPath();
    UpdateLaserVisuals();
}
/// <summary>
/// 设置最大反射次数（由技能等级决定）
/// </summary>
/// <param name="level">技能等级： 1=1 次反射, 3=2 次反射, 5=3 次反射</param>
public void SetReflectionLevel(int level)
{

```

```
if (level <= 0)
{
    maxReflections = 0;
}
else if (level <= 2)
{
    maxReflections = 1;
}
else if (level <= 4)
{
    maxReflections = 2;
}
else
{
    maxReflections = 3;
}
if (showDebugLogs)
{
    Debug.Log($"[LaserController] 反射等级设置 - 技能等级: {level}, 最大反射次数: {maxReflections}");
}
CalculateLaserPath();
UpdateLaserVisuals();
}
/// <summary>
/// 直接设置最大反射次数
/// </summary>
public void SetMaxReflections(int count)
{
    maxReflections = Mathf.Clamp(count, 0, 3);
    CalculateLaserPath();
    UpdateLaserVisuals();
}
/// <summary>
/// 获取当前旋转角度
/// </summary>
public float GetCurrentAngle()
{
    return currentAngle;
}
/// <summary>
/// 获取激光起点世界坐标
/// </summary>
public Vector3 GetStartPoint()
{
    return laserPoints.Count > 0 ? laserPoints[0] : Vector3.zero;
}
/// <summary>
/// 获取激光终点世界坐标
/// </summary>
```

```
public Vector3 GetEndPoint()
{
return laserPoints.Count > 0 ? laserPoints[laserPoints.Count - 1] : Vector3.zero;
}
/// <summary>
/// 获取所有激光路径点
/// </summary>
public List<Vector3> GetLaserPoints()
{
return new List<Vector3>(laserPoints);
}
/// <summary>
/// 获取当前反射次数
/// </summary>
public int GetCurrentReflectionCount()
{
return currentReflectionCount;
}
/// <summary>
/// 激光是否击中敌人
/// </summary>
public bool HasHitEnemy()
{
return hitEnemy;
}
/// <summary>
/// 获取激光方向（第一段）
/// </summary>
public Vector3 GetLaserDirection()
{
if (laserPoints.Count < 2) return Vector3.up;
return (laserPoints[1] - laserPoints[0]).normalized;
}
#endregion
#region Debug
private void OnDrawGizmos()
{
if (!showDebugGizmos) return;
if (laserPoints == null || laserPoints.Count < 2) return;
// 绘制激光路径
for (int i = 0; i < laserPoints.Count; i++)
{
// 绘制点
if (i == 0)
{
// 起点（绿色）
Gizmos.color = Color.green;
}
else if (i == laserPoints.Count - 1)
{

```

```

// 终点（红色或橙色）
Gizmos.color = hitEnemy ? Color.red : new Color(1f, 0.5f, 0f);
}
else
{
// 反射点（蓝色）
Gizmos.color = Color.blue;
}
Gizmos.DrawWireSphere(laserPoints[i], 0.3f);
// 绘制线段
if (i < laserPoints.Count - 1)
{
Gizmos.color = Color.yellow;
Gizmos.DrawLine(laserPoints[i], laserPoints[i + 1]);
}
}
// 绘制反射次数标签位置
if (currentReflectionCount > 0)
{
Gizmos.color = Color.cyan;
for (int i = 1; i < laserPoints.Count - 1; i++)
{
Gizmos.DrawWireCube(laserPoints[i], Vector3.one * 0.2f);
}
}
}
#endregion
#region Helper Structs
/// <summary>
/// Raycast 结果数据结构
/// </summary>
private struct RaycastResult
{
public bool hitSomething;
public bool hitEnemy;
public bool hitWall;
public Vector3 hitPoint;
public Vector3 hitNormal;
public float hitDistance;
public Collider2D hitCollider;
}
#endregion
}
}
using UnityEngine;
/// <summary>
/// 简化版激光测试脚本
/// 用于测试击退效果，独立于主游戏系统
///
/// 使用方法：

```

```

/// 1. 将此脚本挂载到激光 Pivot 或 Turret 上
/// 2. 将 laserOrigin 设置为激光发射点
/// 3. 运行游戏，激光会自动检测并击退 Enemy Layer 上的物体
/// </summary>
public class LaserKnockbackTester : MonoBehaviour
{
//

```

```

// Inspector 配置
//

```

```

[Header("激光设置")]
[Tooltip("激光发射点（如果为空则使用自身 Transform）")]
[SerializeField] private Transform laserOrigin;
[Tooltip("激光最大长度")]
[SerializeField] private float maxLength = 15f;
[Tooltip("激光宽度（仅用于 Gizmo 显示）")]
[SerializeField] private float laserWidth = 0.5f;
[Header("伤害设置")]
[Tooltip("每秒伤害 (DPS)")]
[SerializeField] private float dps = 100f;
[Tooltip("伤害判定间隔（秒）")]
[SerializeField] private float tickRate = 0.1f;
[Header("击退设置")]
[Tooltip("基础击退力")]
[SerializeField] private float baseKnockbackForce = 50f;
[Tooltip("击退力模式")]
[SerializeField] private ForceMode2D forceMode = ForceMode2D.Force;
[Header("Layer 设置")]
[Tooltip("检测的 Layer")]
[SerializeField] private LayerMask targetLayer;
[Header("调试")]
[SerializeField] private bool showDebugGizmos = true;
[SerializeField] private bool showDebugLog = true;
//

```

```

// 运行时数据
//

```

```

private float lastTickTime;
private RaycastHit2D currentHit;
private float currentHitDistance;
//

```

```

// Unity 生命周期

```

```
//
```

```
private void Awake()
{
    if (laserOrigin == null)
    {
        laserOrigin = transform;
    }
    // 自动设置 Enemy Layer
    if (targetLayer == 0)
    {
        targetLayer = LayerMask.GetMask("Enemy");
        Debug.Log("[LaserKnockbackTester] 自动设置 targetLayer 为 'Enemy'");
    }
}

private void Update()
{
    // 执行 Raycast 检测
    PerformRaycast();
    // 处理伤害和击退
    ProcessDamageAndKnockback();
}

//
```

```
// 核心逻辑
//
```

```
private void PerformRaycast()
{
    Vector2 origin = laserOrigin.position;
    Vector2 direction = laserOrigin.up; // 假设激光朝上
    currentHit = Physics2D.Raycast(origin, direction, maxLength, targetLayer);
    if (currentHit.collider != null)
    {
        currentHitDistance = currentHit.distance;
    }
    else
    {
        currentHitDistance = maxLength;
    }
}

private void ProcessDamageAndKnockback()
{
    // 检查 Tick 间隔
    if (Time.time - lastTickTime < tickRate)
        return;
    lastTickTime = Time.time;
```

```
// 如果没有击中任何东西
if (currentHit.collider == null)
return;
// 计算伤害
float damage = dps * tickRate;
// 计算击退力方向（从激光原点指向目标）
Vector2 knockbackDirection = (currentHit.point - (Vector2)laserOrigin.position).normalized;
Vector2 knockbackForce = knockbackDirection * baseKnockbackForce;
// 尝试获取 KnockbackTestCube 组件
var testCube = currentHit.collider.GetComponent<KnockbackTestCube>();
if (testCube != null)
{
testCube.TakeDamage(damage, knockbackForce);
if (showDebugLog)
{
Debug.Log($"[LaserKnockbackTester] 击中 KnockbackTestCube" +
$"\n 伤害: {damage:F1}" +
$"\n 击退方向: {knockbackDirection}" +
$"\n 击退力: {knockbackForce.magnitude:F2}");
}
return;
}
// 尝试获取 EnemyBlob 组件（兼容现有系统）
var enemyBlob = currentHit.collider.GetComponent<LightVsDecay.Logic.Enemy.EnemyBlob>();
if (enemyBlob != null)
{
enemyBlob.TakeDamage(damage, knockbackForce);
if (showDebugLog)
{
Debug.Log($"[LaserKnockbackTester] 击中 EnemyBlob: {enemyBlob.name}");
}
return;
}
// 如果都没有，尝试直接操作 Rigidbody2D
var rb = currentHit.collider.GetComponent<Rigidbody2D>();
if (rb != null)
{
rb.AddForce(knockbackForce, forceMode);
if (showDebugLog)
{
Debug.Log($"[LaserKnockbackTester] 直接击退 Rigidbody2D: {rb.name}");
}
}
}
}
//
```

```
// 调试可视化
//
```

```

private void OnDrawGizmos()
{
    if (!showDebugGizmos) return;
    Transform origin = laserOrigin != null ? laserOrigin : transform;
    Vector3 start = origin.position;
    Vector3 direction = origin.up;
    // 绘制激光线
    if (Application.isPlaying)
    {
        // 运行时：显示实际检测结果
        Gizmos.color = currentHit.collider != null ? Color.red : Color.green;
        Gizmos.DrawLine(start, start + direction * currentHitDistance);
        // 绘制击中点
        if (currentHit.collider != null)
        {
            Gizmos.color = Color.yellow;
            Gizmos.DrawWireSphere(currentHit.point, 0.3f);
            // 绘制击退方向
            Gizmos.color = Color.magenta;
            Vector3 knockbackDir = (currentHit.point - (Vector2)start).normalized;
            //Gizmos.DrawLine(currentHit.point, currentHit.point + knockbackDir * 2f);
        }
    }
    else
    {
        // 编辑器模式：显示最大长度
        Gizmos.color = Color.cyan;
        Gizmos.DrawLine(start, start + direction * maxLength);
    }
    // 绘制激光宽度（矩形边界）
    Gizmos.color = new Color(0f, 1f, 1f, 0.3f);
    Vector3 right = origin.right * laserWidth / 2f;
    Vector3 end = start + direction * (Application.isPlaying ? currentHitDistance : maxLength);
    Gizmos.DrawLine(start + right, end + right);
    Gizmos.DrawLine(start - right, end - right);
}
//

```

```

// 编辑器调试
//

```

```

private void OnGUI()
{
    if (!showDebugLog || !Application.isPlaying) return;
    GUILayout.BeginArea(new Rect(10, 10, 300, 150));
    GUI.color = Color.white;
    GUILayout.Label("=== Laser Knockback Tester ===");
}

```

```

GUILayout.Label($"击中目标: {(currentHit.collider != null ? currentHit.collider.name : "无")}");
GUILayout.Label($"击中距离: {currentHitDistance:F2}");
GUILayout.Label($"DPS: {dps} | 单次伤害: {dps * tickRate:F1}");
GUILayout.Label($"击退力: {baseKnockbackForce}");
GUILayout.Label($"力模式: {forceMode}");
GUILayout.EndArea();
}
}
using UnityEngine;
namespace NeonGamblingTower.Laser
{
/// <summary>
/// 激光测试辅助工具 - Step 2
/// 提供运行时按钮和滑块快速测试激光参数和反射功能
/// </summary>
public class LaserTestHelper : MonoBehaviour
{
[Header("=== 控制器引用 ===")]
[SerializeField] private LaserLineRendererController laserController;
[Header("=== 基础测试参数 ===")]
[Range(5f, 30f)]
[SerializeField] private float testLength = 19f;
[Range(0.1f, 3f)]
[SerializeField] private float testWidth = 0.5f;
[Range(-90f, 90f)]
[SerializeField] private float testAngle = 0f;
[Header("=== 反射测试 ===")]
[Range(0, 3)]
[SerializeField] private int testMaxReflections = 0;
[Header("=== 自动测试 ===")]
[SerializeField] private bool autoRotate = false;
[SerializeField] private float autoRotateSpeed = 30f;
[SerializeField] private bool pulseWidth = false;
[SerializeField] private float pulseSpeed = 2f;
[SerializeField] private float pulseMinWidth = 0.2f;
[SerializeField] private float pulseMaxWidth = 1.5f;
[Header("=== 运行时信息（只读） ===")]
[SerializeField] private int currentReflections = 0;
[SerializeField] private bool hasHitEnemy = false;
[SerializeField] private Vector3 laserEndPoint;
private float previousLength;
private float previousWidth;
private float previousAngle;
private int previousReflections;
private void Start()
{
if (laserController == null)
{
laserController = GetComponent<LaserLineRendererController>();
}
}
}

```

```
previousLength = testLength;
previousWidth = testWidth;
previousAngle = testAngle;
previousReflections = testMaxReflections;
}
private void Update()
{
// 检测 Inspector 中参数变化
CheckParameterChanges();
// 自动旋转测试
if (autoRotate)
{
AutoRotateTest();
}
// 宽度脉冲测试
if (pulseWidth)
{
PulseWidthTest();
}
// 更新运行时信息
UpdateRuntimeInfo();
}
/// <summary>
/// 检测参数变化并应用
/// </summary>
private void CheckParameterChanges()
{
if (laserController == null) return;
// 长度变化
if (!Mathf.Approximately(testLength, previousLength))
{
laserController.SetLength(testLength);
previousLength = testLength;
Debug.Log($"[LaserTest] 长度设置为: {testLength}");
}
// 宽度变化
if (!Mathf.Approximately(testWidth, previousWidth))
{
laserController.SetWidth(testWidth);
previousWidth = testWidth;
Debug.Log($"[LaserTest] 宽度设置为: {testWidth}");
}
// 角度变化（仅当不自动旋转时）
if (!autoRotate && !Mathf.Approximately(testAngle, previousAngle))
{
laserController.SetRotation(testAngle);
previousAngle = testAngle;
Debug.Log($"[LaserTest] 角度设置为: {testAngle}");
}
// 反射次数变化
```

```

if (testMaxReflections != previousReflections)
{
    laserController.SetMaxReflections(testMaxReflections);
    previousReflections = testMaxReflections;
    Debug.Log($"[LaserTest] 最大反射次数设置为: {testMaxReflections}");
}
}

/// <summary>
/// 自动旋转测试
/// </summary>
private void AutoRotateTest()
{
    if (laserController == null) return;
    testAngle = Mathf.PingPong(Time.time * autoRotateSpeed, 180f) - 90f;
    laserController.SetRotation(testAngle);
}

/// <summary>
/// 宽度脉冲测试
/// </summary>
private void PulseWidthTest()
{
    if (laserController == null) return;
    float t = (Mathf.Sin(Time.time * pulseSpeed) + 1f) / 2f;
    testWidth = Mathf.Lerp(pulseMinWidth, pulseMaxWidth, t);
    laserController.SetWidth(testWidth);
}

/// <summary>
/// 更新运行时信息
/// </summary>
private void UpdateRuntimeInfo()
{
    if (laserController == null) return;
    currentReflections = laserController.GetCurrentReflectionCount();
    hasHitEnemy = laserController.HasHitEnemy();
    laserEndPoint = laserController.GetEndPoint();
}

/// <summary>
/// 重置为默认值
/// </summary>
[ContextMenu("重置参数")]
public void ResetParameters()
{
    testLength = 19f;
    testWidth = 0.5f;
    testAngle = 0f;
    testMaxReflections = 0;
    autoRotate = false;
    pulseWidth = false;
    if (laserController != null)
    {

```

```
laserController.SetLength(testLength);
laserController.SetWidth(testWidth);
laserController.SetRotation(testAngle);
laserController.SetMaxReflections(testMaxReflections);
}
Debug.Log("[LaserTest] 参数已重置");
}
/// <summary>
/// 测试最大长度
/// </summary>
[ContextMenu("测试 - 最大长度")]
public void TestMaxLength()
{
    testLength = 30f;
    if (laserController != null)
    {
        laserController.SetLength(testLength);
    }
}
/// <summary>
/// 测试最大宽度
/// </summary>
[ContextMenu("测试 - 最大宽度")]
public void TestMaxWidth()
{
    testWidth = 3f;
    if (laserController != null)
    {
        laserController.SetWidth(testWidth);
    }
}
/// <summary>
/// 测试 1 次反射
/// </summary>
[ContextMenu("测试 - 1 次反射")]
public void TestReflection1()
{
    testMaxReflections = 1;
    if (laserController != null)
    {
        laserController.SetMaxReflections(testMaxReflections);
    }
    Debug.Log("[LaserTest] 设置 1 次反射");
}
/// <summary>
/// 测试 2 次反射
/// </summary>
[ContextMenu("测试 - 2 次反射")]
public void TestReflection2()
{

```

```
testMaxReflections = 2;
if (laserController != null)
{
    laserController.SetMaxReflections(testMaxReflections);
}
Debug.Log("[LaserTest] 设置 2 次反射");
}
/// <summary>
/// 测试 3 次反射
/// </summary>
[ContextMenu("测试 - 3 次反射")]
public void TestReflection3()
{
    testMaxReflections = 3;
    if (laserController != null)
    {
        laserController.SetMaxReflections(testMaxReflections);
    }
    Debug.Log("[LaserTest] 设置 3 次反射");
}
/// <summary>
/// 关闭反射
/// </summary>
[ContextMenu("测试 - 关闭反射")]
public void TestReflectionOff()
{
    testMaxReflections = 0;
    if (laserController != null)
    {
        laserController.SetMaxReflections(testMaxReflections);
    }
    Debug.Log("[LaserTest] 关闭反射");
}
}
}
using UnityEngine;
/// <summary>
/// 击退测试方块
/// 用于验证激光击退效果是否正常工作
///
/// 使用方法:
/// 1. 创建一个 2D Sprite (方块或圆形都可以)
/// 2. 添加 Rigidbody2D 组件
/// 3. 添加 BoxCollider2D 或 CircleCollider2D 组件
/// 4. 将该物体的 Layer 设置为 "Enemy"
/// 5. 挂载此脚本
/// 6. 运行游戏, 用激光照射方块
/// </summary>
[RequireComponent(typeof(Rigidbody2D))]
[RequireComponent(typeof(Collider2D))]
```

```

public class KnockbackTestCube : MonoBehaviour
{
//

```

```

// Inspector 配置
//

```

```

[Header("物理设置")]
[Tooltip("方块质量（越大越难推动）")]
[SerializeField] private float mass = 1.0f;
[Tooltip("线性阻力（越大停下越快）")]
[SerializeField] private float linearDrag = 2.0f;
[Header("击退设置")]
[Tooltip("击退力倍率（调试用）")]
[SerializeField] private float knockbackMultiplier = 1.0f;
[Tooltip("使用 Impulse 模式（瞬间冲击）还是 Force 模式（持续推力）")]
[SerializeField] private bool useImpulseMode = true;
[Header("视觉反馈")]
[Tooltip("受击时的颜色")]
[SerializeField] private Color hitColor = Color.red;
[Tooltip("颜色恢复时间")]
[SerializeField] private float colorRecoveryTime = 0.2f;
[Header("调试信息")]
[SerializeField] private bool showDebugLog = true;
//

```

```

// 运行时数据
//

```

```

private Rigidbody2D rb;
private SpriteRenderer spriteRenderer;
private Color originalColor;
private float lastHitTime;
// 统计数据
private int hitCount = 0;
private float totalDamageReceived = 0f;
//

```

```

// Unity 生命周期
//

```

```

private void Awake()
{
rb = GetComponent<Rigidbody2D>();

```

```

spriteRenderer = GetComponent<SpriteRenderer>();
// 配置 Rigidbody2D
ConfigureRigidbody();
// 保存原始颜色
if (spriteRenderer != null)
{
    originalColor = spriteRenderer.color;
}
// 检查 Layer
if (gameObject.layer != LayerMask.NameToLayer("Enemy"))
{
    Debug.LogWarning($"[KnockbackTestCube] 警告：当前 Layer 不是 'Enemy'! 激光可能无法检测到。当前 Layer: {LayerMask.LayerToName(gameObject.layer)}");
}
}
private void Update()
{
    // 颜色恢复
    UpdateColorRecovery();
}
//

```

```

// 配置
//

```

```

private void ConfigureRigidbody()
{
    rb.gravityScale = 0f;           // 无重力（2D 俯视角）
    rb.mass = mass;
    rb.drag = linearDrag;
    rb.angularDrag = 1f;
    rb.interpolation = RigidbodyInterpolation2D.Interpolate;
    rb.collisionDetectionMode = CollisionDetectionMode2D.Continuous;
    rb.constraints = RigidbodyConstraints2D.FreezeRotation; // 防止旋转
    if (showDebugLog)
    {
        Debug.Log($"[KnockbackTestCube] Rigidbody 配置完成 - Mass: {mass}, Drag: {linearDrag}");
    }
}
//

```

```

// 核心接口：被激光调用
//

```

```

/// <summary>
/// 接收伤害和击退力（与 EnemyBlob.TakeDamage 接口相同）

```

```

/// </summary>
/// <param name="damage">伤害值</param>
/// <param name="knockbackForce">击退力向量</param>
public void TakeDamage(float damage, Vector2 knockbackForce)
{
    hitCount++;
    totalDamageReceived += damage;
    lastHitTime = Time.time;
    // 应用击退力
    ApplyKnockback(knockbackForce);
    // 视觉反馈
    TriggerHitFlash();
    if (showDebugLog)
    {
        Debug.Log($"[KnockbackTestCube] 受击 #{hitCount}" +
            $"\\n 伤害: {damage:F1}" +
            $"\\n 击退力: {knockbackForce} (magnitude: {knockbackForce.magnitude:F2})" +
            $"\\n 当前速度: {rb.velocity} (magnitude: {rb.velocity.magnitude:F2})" +
            $"\\n 累计伤害: {totalDamageReceived:F1}");
    }
}
/// <summary>
/// 应用击退力
/// </summary>
private void ApplyKnockback(Vector2 force)
{
    Vector2 finalForce = force * knockbackMultiplier;
    if (useImpulseMode)
    {
        // Impulse 模式: 瞬间冲击, 适合单次击退
        rb.AddForce(finalForce, ForceMode2D.Impulse);
    }
    else
    {
        // Force 模式: 持续推力, 适合持续激光
        rb.AddForce(finalForce, ForceMode2D.Force);
    }
}
//

```

```

// 视觉反馈
//

```

```

private void TriggerHitFlash()
{
    if (spriteRenderer != null)
    {
        spriteRenderer.color = hitColor;
    }
}

```

```

}
}
private void UpdateColorRecovery()
{
    if (spriteRenderer == null) return;
    float timeSinceHit = Time.time - lastHitTime;
    if (timeSinceHit < colorRecoveryTime)
    {
        float t = timeSinceHit / colorRecoveryTime;
        spriteRenderer.color = Color.Lerp(hitColor, originalColor, t);
    }
    else if (spriteRenderer.color != originalColor)
    {
        spriteRenderer.color = originalColor;
    }
}
//

```

```

// 编辑器调试
//

```

```

private void OnGUI()
{
    if (!showDebugLog || !Application.isPlaying) return;
    // 在物体位置上方显示信息
    Vector3 screenPos = Camera.main.WorldToScreenPoint(transform.position + Vector3.up * 1.5f);
    // 转换为 GUI 坐标 (Y 轴翻转)
    screenPos.y = Screen.height - screenPos.y;
    // 只在屏幕内显示
    if (screenPos.z > 0 && screenPos.x > 0 && screenPos.x < Screen.width && screenPos.y > 0 &&
        screenPos.y < Screen.height)
    {
        GUI.color = Color.yellow;
        GUI.Label(new Rect(screenPos.x - 60, screenPos.y - 60, 200, 100),
            $"击中次数: {hitCount}\n" +
            $"累计伤害: {totalDamageReceived:F0}\n" +
            $"当前速度: {rb.velocity.magnitude:F2}");
    }
}
private void OnDrawGizmosSelected()
{
    // 绘制当前速度向量
    if (Application.isPlaying && rb != null)
    {
        Gizmos.color = Color.green;
        Gizmos.DrawLine(transform.position, transform.position + (Vector3)rb.velocity);
        // 绘制速度箭头
        Gizmos.DrawWireSphere(transform.position + (Vector3)rb.velocity, 0.1f);
    }
}

```

```

}
}
//

```

```

// 重置方法（编辑器测试用）
//

```

```

[ContextMenu("重置位置和状态")]
public void ResetState()
{
    transform.position = Vector3.zero;
    rb.velocity = Vector2.zero;
    rb.angularVelocity = 0f;
    hitCount = 0;
    totalDamageReceived = 0f;
    if (spriteRenderer != null)
    {
        spriteRenderer.color = originalColor;
    }
    Debug.Log("[KnockbackTestCube] 状态已重置");
}
[ContextMenu("测试击退（向上）")]
public void TestKnockbackUp()
{
    TakeDamage(10f, Vector2.up * 50f);
}
[ContextMenu("测试击退（向右）")]
public void TestKnockbackRight()
{
    TakeDamage(10f, Vector2.right * 50f);
}
}
// =====
// MainSceneUIManager.cs
// 文件位置: Assets/Scripts/UI/MainSceneUIManager.cs
// 用途: 主场景 UI 管理器 - 负责面板切换（主界面、科技树、装备）
// =====
using UnityEngine;
using UnityEngine.UI;
using LightVsDecay.Audio;
namespace LightVsDecay.UI
{
    /// <summary>
    /// 主场景 UI 状态枚举
    /// </summary>
    public enum MainSceneState
    {
        Main, // 主界面（默认）
    }
}

```

```

KeJi,        // 科技树界面
ZhuangBei    // 装备界面
}
/// <summary>
/// 主场景 UI 管理器
/// 负责 MainScene 中各面板的切换和按钮状态管理
/// </summary>
public class MainSceneUIManager : MonoBehaviour
{
    //

    // 单例
    //

    public static MainSceneUIManager Instance { get; private set; }
    //

    // Inspector 配置 - 面板引用
    //

    [Header("==== 面板引用 ====")]
    [Tooltip("全局背景（KeJiPanel 显示时隐藏）")]
    [SerializeField] private GameObject globalBackground;
    [Tooltip("主界面面板")]
    [SerializeField] private GameObject mainPanel;
    [Tooltip("科技树面板")]
    [SerializeField] private GameObject keJiPanel;
    [Tooltip("装备面板")]
    [SerializeField] private GameObject zhuangBeiPanel;
    //

    // Inspector 配置 - TopArea 按钮引用
    //

    [Header("==== TopArea 按钮 ====")]
    [Tooltip("设置按钮")]
    [SerializeField] private GameObject settingButton;
    [Tooltip("返回按钮")]
    [SerializeField] private GameObject backButton;
    //

    // Inspector 配置 - BottomArea 按钮引用
    //

```

```
[Header("==== BottomArea 按钮 ====")]
[Tooltip("科技树按钮")]
[SerializeField] private Button keJiButton;
[Tooltip("装备按钮")]
[SerializeField] private Button zhuangBeiButton;
//
```

```
// Inspector 配置 - 返回按钮
//
```

```
[Header("==== 返回按钮引用 ====")]
[Tooltip("返回按钮组件")]
[SerializeField] private Button backButtonComponent;
//
```

```
// Inspector 配置 - 调试
//
```

```
[Header("==== 调试 ====")]
[SerializeField] private bool showDebugInfo = false;
//
```

```
// 运行时状态
//
```

```
private MainSceneState currentState = MainSceneState.Main;
/// <summary>当前 UI 状态</summary>
public MainSceneState CurrentState => currentState;
//
```

```
// Unity 生命周期
//
```

```
private void Awake()
{
// 单例设置
if (Instance != null && Instance != this)
{
Destroy(gameObject);
return;
}
```

```
}
Instance = this;
}
private void Start()
{
SetupButtons();
// 初始化为主界面状态
SwitchToState(MainSceneState.Main);
}
private void OnDestroy()
{
if (Instance == this)
{
Instance = null;
}
}
//

```

```
// 初始化
//

```

```
/// <summary>
/// 设置按钮事件
/// </summary>
private void SetupButtons()
{
// 科技树按钮
if (keJiButton != null)
{
keJiButton.onClick.AddListener(OnKeJiButtonClicked);
}
// 装备按钮
if (zhuangBeiButton != null)
{
zhuangBeiButton.onClick.AddListener(OnZhuangBeiButtonClicked);
}
// 返回按钮
if (backButtonComponent != null)
{
backButtonComponent.onClick.AddListener(OnBackButtonClicked);
}
}
//

```

```
// 按钮回调
//

```

```
/// <summary>
/// 科技树按钮点击
/// </summary>
private void OnKeJiButtonClicked()
{
    PlayButtonSound();
    SwitchToState(MainSceneState.KeJi);
    if (showDebugInfo)
    {
        Debug.Log("[MainSceneUIManager] 切换到科技树界面");
    }
}

/// <summary>
/// 装备按钮点击
/// </summary>
private void OnZhuangBeiButtonClicked()
{
    PlayButtonSound();
    SwitchToState(MainSceneState.ZhuangBei);
    if (showDebugInfo)
    {
        Debug.Log("[MainSceneUIManager] 切换到装备界面");
    }
}

/// <summary>
/// 返回按钮点击
/// </summary>
private void OnBackButtonClicked()
{
    PlayButtonSound();
    SwitchToState(MainSceneState.Main);
    if (showDebugInfo)
    {
        Debug.Log("[MainSceneUIManager] 返回主界面");
    }
}

//
```

```
// 状态切换
//
```

```
/// <summary>
/// 切换到指定状态
/// </summary>
/// <param name="newState">目标状态</param>
public void SwitchToState(MainSceneState newState)
{
```

```
currentState = newState;
switch (newState)
{
case MainSceneState.Main:
ApplyMainState();
break;
case MainSceneState.KeJi:
ApplyKeJiState();
break;
case MainSceneState.ZhuangBei:
ApplyZhuangBeiState();
break;
}
if (showDebugInfo)
{
Debug.Log($"[MainSceneUIManager] 状态切换: {newState}");
}
}
}
/// <summary>
/// 应用主界面状态
/// </summary>
private void ApplyMainState()
{
// 面板显示
SetActive(globalBackground, true);
SetActive(mainPanel, true);
SetActive(keJiPanel, false);
SetActive(zhuangBeiPanel, false);
// 按钮显示
SetActive(settingButton, true);
SetActive(backButton, false);
}
/// <summary>
/// 应用科技树状态
/// </summary>
private void ApplyKeJiState()
{
// 面板显示
SetActive(globalBackground, false); // KeJiPanel 有自己的背景
SetActive(mainPanel, false);
SetActive(keJiPanel, true);
SetActive(zhuangBeiPanel, false);
// 按钮显示
SetActive(settingButton, false);
SetActive(backButton, true);
}
/// <summary>
/// 应用装备状态
/// </summary>
private void ApplyZhuangBeiState()
```

```
{
// 面板显示
SetActive(globalBackground, true);    // ZhuangBeiPanel 透出全局背景
SetActive(mainPanel, false);
SetActive(keJiPanel, false);
SetActive(zhuangBeiPanel, true);
// 按钮显示
SetActive(settingButton, false);
SetActive(backButton, true);
}
//
```

```
// 公共接口
//
```

```
/// <summary>
/// 显示科技树界面（供外部调用）
/// </summary>
public void ShowKeJiPanel()
{
    SwitchToState(MainSceneState.KeJi);
}
/// <summary>
/// 显示装备界面（供外部调用）
/// </summary>
public void ShowZhuangBeiPanel()
{
    SwitchToState(MainSceneState.ZhuangBei);
}
/// <summary>
/// 返回主界面（供外部调用）
/// </summary>
public void BackToMain()
{
    SwitchToState(MainSceneState.Main);
}
/// <summary>
/// 检查是否在主界面
/// </summary>
public bool IsInMainState()
{
    return currentState == MainSceneState.Main;
}
//
```

```
// 辅助方法
//
```

```

/// <summary>
/// 安全设置 GameObject 显示状态
/// </summary>
private void SetActive(GameObject obj, bool active)
{
    if (obj != null)
    {
        obj.SetActive(active);
    }
}
/// <summary>
/// 播放按钮音效
/// </summary>
private void PlayButtonSound()
{
    if (AudioManager.Instance != null)
    {
        AudioManager.Instance.PlayButtonClick();
    }
}
}
}
}
}
// =====
// UIManager.cs
// 文件位置: Assets/Scripts/UI/UIManager.cs
// 用途: 统一管理所有 UI 面板的显示/隐藏
// =====
using LightVsDecay.Audio;
using LightVsDecay.Core;
using LightVsDecay.UI.Panels;
using UnityEngine;
namespace LightVsDecay.UI
{
    /// <summary>
    /// UI 管理器 (单例)
    /// 挂载在 Canvas 上, 统一控制所有弹窗面板的显示/隐藏
    /// 各面板控制器只负责业务逻辑, 不负责显隐
    /// </summary>
    public class UIManager : Singleton<UIManager>
    {
        //

```

```

// Inspector 配置 - 面板引用
//

```

```

[Header("面板引用")]

```

```

[Tooltip("结算面板")]
[SerializeField] private GameObject settlementPanel;
[Tooltip("复活面板")]
[SerializeField] private GameObject revivePanel;
[Tooltip("暂停面板")]
[SerializeField] private GameObject pausePanel;
[Tooltip("技能选择面板")] // 【新增】
[SerializeField] private GameObject skillChoosePanel;
//

```

```

// Inspector 配置 - 面板控制器引用
//

```

```

[Header("面板控制器")]
[SerializeField] private SettlementPanel settlementController;
[SerializeField] private SkillChooseOnePanel skillChooseController; // 【新增】
[Header("调试")]
[SerializeField] private bool showDebugInfo = false;
//

```

```

// 运行时状态
//

```

```

private GameObject currentActivePanel = null;
//

```

```

// Unity 生命周期
//

```

```

protected override void OnSingletonAwake()
{
// 初始化：隐藏所有面板
HideAllPanels();
}
private void OnEnable()
{
// 订阅游戏事件
Core.GameEvents.OnGameVictory += OnGameVictory;
Core.GameEvents.OnGameDefeat += OnGameDefeat;
Core.GameEvents.OnGamePaused += OnGamePaused;
Core.GameEvents.OnGameResumed += OnGameResumed;
if (showDebugInfo)
{
Debug.Log("[UIManager] 事件已订阅");
}
}

```

```
}  
}  
private void OnDisable()  
{  
    // 取消订阅  
    Core.GameEvents.OnGameVictory -= OnGameVictory;  
    Core.GameEvents.OnGameDefeat -= OnGameDefeat;  
    Core.GameEvents.OnGamePaused -= OnGamePaused;  
    Core.GameEvents.OnGameResumed -= OnGameResumed;  
    if (showDebugInfo)  
    {  
        Debug.Log("[UIManager] 事件已取消订阅");  
    }  
}  
private void OnSingletonDestroy()  
{  
}  
//  
  
// 事件回调  
//  
  
private void OnGameVictory()  
{  
    if (showDebugInfo)  
    {  
        Debug.Log("[UIManager] 收到胜利事件，显示结算面板");  
    }  
    ShowSettlementPanel(true);  
}  
private void OnGameDefeat()  
{  
    if (showDebugInfo)  
    {  
        Debug.Log("[UIManager] 收到失败事件，显示结算面板");  
    }  
    ShowSettlementPanel(false);  
}  
private void OnGamePaused()  
{  
    if (showDebugInfo)  
    {  
        Debug.Log("[UIManager] 收到暂停事件");  
    }  
    ShowPausePanel();  
}  
private void OnGameResumed()  
{
```

```
if (showDebugInfo)
{
    Debug.Log("[UIManager] 收到恢复事件");
}
HidePausePanel();
}
//

```

```
// 面板控制 - 结算面板
//

```

```
/// <summary>
/// 显示结算面板
/// </summary>
/// <param name="isVictory">是否胜利</param>
public void ShowSettlementPanel(bool isVictory)
{
    // 【新增】停止激光循环音效
    if (AudioManager.Instance != null)
    {
        AudioManager.Instance.StopLaserLoop();
    }
    if (settlementPanel == null)
    {
        Debug.LogWarning("[UIManager] settlementPanel 未设置! ");
        return;
    }
    // 隐藏其他面板
    HideAllPanels();
    // 显示结算面板
    settlementPanel.SetActive(true);
    currentActivePanel = settlementPanel;
    // 通知控制器显示内容
    if (settlementController != null)
    {
        settlementController.Show(isVictory);
    }
    if (showDebugInfo)
    {
        Debug.Log($"[UIManager] 结算面板已显示 (胜利: {isVictory})");
    }
}
/// <summary>
/// 隐藏结算面板
/// </summary>
public void HideSettlementPanel()
{
    if (settlementPanel != null)
```

```
{
settlementPanel.SetActive(false);
if (currentActivePanel == settlementPanel)
{
currentActivePanel = null;
}
}
}
//
```

```
// 面板控制 - 复活面板
```

```
//
```

```
/// <summary>
/// 显示复活面板
/// </summary>
public void ShowRevivePanel()
{
if (revivePanel == null)
{
Debug.LogWarning("[UIManager] revivePanel 未设置!");
return;
}
revivePanel.SetActive(true);
currentActivePanel = revivePanel;
if (showDebugInfo)
{
Debug.Log("[UIManager] 复活面板已显示");
}
}
/// <summary>
/// 隐藏复活面板
/// </summary>
public void HideRevivePanel()
{
if (revivePanel != null)
{
revivePanel.SetActive(false);
if (currentActivePanel == revivePanel)
{
currentActivePanel = null;
}
}
}
//
```

```
// 面板控制 - 暂停面板
```

```
//  
  
/// <summary>  
/// 显示暂停面板  
/// </summary>  
public void ShowPausePanel()  
{  
    // 【新增】停止激光循环音效  
    if (AudioManager.Instance != null)  
    {  
        AudioManager.Instance.StopLaserLoop();  
    }  
    if (pausePanel == null) return;  
    pausePanel.SetActive(true);  
    currentActivePanel = pausePanel;  
    if (showDebugInfo)  
    {  
        Debug.Log("[UIManager] 暂停面板已显示");  
    }  
}  
/// <summary>  
/// 隐藏暂停面板  
/// </summary>  
public void HidePausePanel()  
{  
    if (pausePanel != null)  
    {  
        pausePanel.SetActive(false);  
        if (currentActivePanel == pausePanel)  
        {  
            currentActivePanel = null;  
        }  
    }  
}  
/// <summary>  
/// 显示技能选择面板  
/// </summary>  
public void ShowSkillChoosePanel(int level)  
{  
    if (skillChoosePanel == null)  
    {  
        Debug.LogWarning("[UIManager] skillChoosePanel 未设置! ");  
        return;  
    }  
    // 先隐藏其他面板  
    if (settlementPanel != null) settlementPanel.SetActive(false);  
    if (pausePanel != null) pausePanel.SetActive(false);  
    // 显示技能选择面板  
    skillChoosePanel.SetActive(true);
```

```
currentActivePanel = skillChoosePanel;
// 【修改】自动获取控制器并调用 Show
if (skillChooseController == null)
{
    skillChooseController = skillChoosePanel.GetComponent<SkillChooseOnePanel>();
}
if (skillChooseController != null)
{
    skillChooseController.Show(level);
}
else
{
    Debug.LogError("[UIManager] SkillChooseOnePanel 组件未找到!");
}
if (showDebugInfo)
{
    Debug.Log($"[UIManager] 技能选择面板已显示 Lv.{level}");
}
}
}
/// <summary>
/// 隐藏技能选择面板
/// </summary>
public void HideSkillChoosePanel()
{
    if (skillChoosePanel != null)
    {
        skillChoosePanel.SetActive(false);
        if (currentActivePanel == skillChoosePanel)
        {
            currentActivePanel = null;
        }
    }
}
//
```

```
// 通用方法
```

```
//
```

```
/// <summary>
/// 隐藏所有面板
/// </summary>
public void HideAllPanels()
{
    if (settlementPanel != null) settlementPanel.SetActive(false);
    if (revivePanel != null) revivePanel.SetActive(false);
    if (pausePanel != null) pausePanel.SetActive(false);
    if (skillChoosePanel != null) skillChoosePanel.SetActive(false);
    currentActivePanel = null;
```

```

}
/// <summary>
/// 是否有面板正在显示
/// </summary>
public bool IsAnyPanelActive()
{
return currentActivePanel != null && currentActivePanel.activeSelf;
}
/// <summary>
/// 获取当前激活的面板
/// </summary>
public GameObject GetActivePanel()
{
return currentActivePanel;
}
//

```

```

// 调试
//

```

```

#if UNITY_EDITOR
private void OnGUI()
{
if (!showDebugInfo) return;
GUILayout.BeginArea(new Rect(Screen.width - 200, 150, 190, 150));
GUILayout.Label("== UIManager ==");
GUILayout.Label($"Active Panel: {(currentActivePanel != null ? currentActivePanel.name :
"None")}");
GUILayout.Space(5);
if (GUILayout.Button("Show Victory"))
{
ShowSettlementPanel(true);
}
if (GUILayout.Button("Show Defeat"))
{
ShowSettlementPanel(false);
}
if (GUILayout.Button("Hide All"))
{
HideAllPanels();
}
GUILayout.EndArea();
}
#endif
}
}
// =====
// PlayerDamageTextHandler.cs

```

```
// 文件位置: Assets/Scripts/UI/FloatingText/PlayerDamageTextHandler.cs
```

```
// 用途: 玩家受击/恢复飘字事件处理器
```

```
// =====
```

```
using UnityEngine;
```

```
using LightVsDecay.Core;
```

```
namespace LightVsDecay.UI.FloatingText
```

```
{
```

```
/// <summary>
```

```
/// 玩家受击飘字事件处理器
```

```
/// 监听 GameEvents 并调用 FloatingTextManager
```

```
/// 挂载到 GameScene 的 UI 管理器上
```

```
/// </summary>
```

```
public class PlayerDamageTextHandler : MonoBehaviour
```

```
{
```

```
[Header("调试")]
```

```
[SerializeField] private bool showDebugInfo = false;
```

```
//
```

```
// Unity 生命周期
```

```
//
```

```
private void OnEnable()
```

```
{
```

```
GameEvents.OnPlayerHealthDamaged += OnHealthDamaged;
```

```
GameEvents.OnPlayerShieldDamaged += OnShieldDamaged;
```

```
GameEvents.OnPlayerHealthRestored += OnHealthRestored;
```

```
GameEvents.OnPlayerShieldRestored += OnShieldRestored;
```

```
if (showDebugInfo)
```

```
{
```

```
Debug.Log("[PlayerDamageTextHandler] 事件订阅完成");
```

```
}
```

```
}
```

```
private void OnDisable()
```

```
{
```

```
GameEvents.OnPlayerHealthDamaged -= OnHealthDamaged;
```

```
GameEvents.OnPlayerShieldDamaged -= OnShieldDamaged;
```

```
GameEvents.OnPlayerHealthRestored -= OnHealthRestored;
```

```
GameEvents.OnPlayerShieldRestored -= OnShieldRestored;
```

```
}
```

```
//
```

```
// 事件回调
```

```
//
```

```
private void OnHealthDamaged(int damage, Vector3 position)
```

```
{
```

```

if (damage <= 0) return;
if (FloatingTextManager.Instance != null)
{
    FloatingTextManager.Instance.ShowPlayerHealthDamage(position, damage);
}
if (showDebugInfo)
{
    Debug.Log($"[PlayerDamageTextHandler] 血量受伤飘字: -{damage}");
}
}

private void OnShieldDamaged(int damage, Vector3 position)
{
    if (damage <= 0) return;
    if (FloatingTextManager.Instance != null)
    {
        FloatingTextManager.Instance.ShowPlayerShieldDamage(position, damage);
    }
    if (showDebugInfo)
    {
        Debug.Log($"[PlayerDamageTextHandler] 护盾受伤飘字: -{damage}");
    }
}

private void OnHealthRestored(int amount, Vector3 position)
{
    if (amount <= 0) return;
    if (FloatingTextManager.Instance != null)
    {
        FloatingTextManager.Instance.ShowPlayerHealthRestore(position, amount);
    }
    if (showDebugInfo)
    {
        Debug.Log($"[PlayerDamageTextHandler] 血量恢复飘字: +{amount}");
    }
}

private void OnShieldRestored(int amount, Vector3 position)
{
    if (amount <= 0) return;
    if (FloatingTextManager.Instance != null)
    {
        FloatingTextManager.Instance.ShowPlayerShieldRestore(position, amount);
    }
    if (showDebugInfo)
    {
        Debug.Log($"[PlayerDamageTextHandler] 护盾恢复飘字: +{amount}");
    }
}
}
}
}
}
}
// =====
// DroneRewardText.cs

```

```
// 文件位置: Assets/Scripts/UI/TacticalDrop/DroneRewardText.cs
// 用途: 无人机奖励飘字组件 (专用动画: 弹出→漂浮→消失)
```

```
// =====
```

```
using UnityEngine;
using UnityEngine.UI;
using TMPro;
using DG.Tweening;
namespace LightVsDecay.UI.FloatingText.TacticalDrop
{
    /// <summary>
    /// 无人机奖励飘字组件
    /// 动画阶段:
    /// 1. 弹出 (0s-0.1s): 缩放 0% → 120%
    /// 2. 回弹 (0.1s-0.2s): 缩放 120% → 100%
    /// 3. 漂浮 (0.2s-0.8s): Y 轴 +50 像素, Ease-Out
    /// 4. 消失 (0.6s-0.8s): 透明度 100% → 0%
    /// </summary>
    public class DroneRewardText : MonoBehaviour
    {
        //
```

```
// 组件引用 (单行模式: 补给/问号)
//
```

```
[Header("单行模式组件")]
[Tooltip("图标 Image")]
[SerializeField] private Image iconImage;
[Tooltip("文本 TMP")]
[SerializeField] private TextMeshProUGUI textMesh;
//
```

```
// 组件引用 (双行模式: 契约)
//
```

```
[Header("双行模式组件 (契约箱专用)")]
[Tooltip("代价行图标")]
[SerializeField] private Image costIconImage;
[Tooltip("代价行文本")]
[SerializeField] private TextMeshProUGUI costTextMesh;
[Tooltip("收益行图标")]
[SerializeField] private Image gainIconImage;
[Tooltip("收益行文本")]
[SerializeField] private TextMeshProUGUI gainTextMesh;
//
```

```
// 通用组件
```

```
//
```

```
[Header("通用组件")]
[Tooltip("CanvasGroup（用于淡出）")]
[SerializeField] private CanvasGroup canvasGroup;
[Tooltip("RectTransform")]
[SerializeField] private RectTransform rectTransform;
//
```

```
// 动画参数
```

```
//
```

```
[Header("动画参数")]
[Tooltip("弹出时间")]
[SerializeField] private float popDuration = 0.1f;
[Tooltip("弹出缩放")]
[SerializeField] private float popScale = 1.2f;
[Tooltip("回弹时间")]
[SerializeField] private float bounceDuration = 0.1f;
[Tooltip("漂浮时间")]
[SerializeField] private float floatDuration = 0.6f;
[Tooltip("漂浮距离（像素）")]
[SerializeField] private float floatDistance = 50f;
[Tooltip("淡出开始时间（相对于漂浮开始）")]
[SerializeField] private float fadeStartDelay = 0.4f;
[Tooltip("淡出时间")]
[SerializeField] private float fadeDuration = 0.2f;
//
```

```
// 运行时状态
```

```
//
```

```
private bool isPlaying = false;
private Sequence animSequence;
//
```

```
// 属性
```

```
//
```

```
public bool IsPlaying => isPlaying;
//
```

```
// Unity 生命周期
```

```
//
```

```
private void Awake()
{
    if (rectTransform == null)
    {
        rectTransform = GetComponent<RectTransform>();
    }
    if (canvasGroup == null)
    {
        canvasGroup = GetComponent<CanvasGroup>();
        if (canvasGroup == null)
        {
            canvasGroup = gameObject.AddComponent<CanvasGroup>();
        }
    }
}
private void OnDestroy()
{
    animSequence?.Kill();
}
//
```

```
// 公共接口 - 单行模式（补给/问号）
```

```
//
```

```
/// <summary>
/// 播放单行飘字（补给/问号无人机）
/// </summary>
/// <param name="worldPosition">世界坐标</param>
/// <param name="icon">图标 Sprite</param>
/// <param name="text">文本（如 "+100"）</param>
/// <param name="textColor">文本颜色</param>
/// <param name="completeCallback">完成回调</param>
public void PlaySingle(
    Vector3 worldPosition,
    Sprite icon,
    string text,
    Color textColor)
{
    if (isPlaying) return;
    // 隐藏双行组件
    SetDualRowActive(false);
    // 设置单行内容
    if (iconImage != null)
```

```

{
iconImage.gameObject.SetActive(true);
iconImage.sprite = icon;
}
if (textMesh != null)
{
textMesh.gameObject.SetActive(true);
textMesh.text = text;
textMesh.color = textColor;
}
// 设置位置
SetupPosition(worldPosition);
// 播放动画
PlayAnimation();
}
//

```

```

// 公共接口 - 双行模式（契约）
//

```

```

/// <summary>
/// 播放双行飘字（契约无人机）
/// </summary>
/// <param name="worldPosition">世界坐标</param>
/// <param name="costIcon">代价图标</param>
/// <param name="costText">代价文本（如 "HP -100"）</param>
/// <param name="costColor">代价颜色（红色）</param>
/// <param name="gainIcon">收益图标</param>
/// <param name="gainText">收益文本（如 "ATK +10%"）</param>
/// <param name="gainColor">收益颜色（绿色）</param>
/// <param name="completeCallback">完成回调</param>
public void PlayDual(
Vector3 worldPosition,
Sprite costIcon,
string costText,
Color costColor,
Sprite gainIcon,
string gainText,
Color gainColor)
{
if (isPlaying) return;
// 隐藏单行组件
SetSingleRowActive(false);
// 设置代价行
if (costIconImage != null)
{
costIconImage.gameObject.SetActive(true);
costIconImage.sprite = costIcon;

```

```

    }
    if (costTextMesh != null)
    {
        costTextMesh.gameObject.SetActive(true);
        costTextMesh.text = costText;
        costTextMesh.color = costColor;
    }
    // 设置收益行
    if (gainIconImage != null)
    {
        gainIconImage.gameObject.SetActive(true);
        gainIconImage.sprite = gainIcon;
    }
    if (gainTextMesh != null)
    {
        gainTextMesh.gameObject.SetActive(true);
        gainTextMesh.text = gainText;
        gainTextMesh.color = gainColor;
    }
    // 设置位置
    SetupPosition(worldPosition);
    // 播放动画
    PlayAnimation();
}
//


```

```

// 私有方法
//


```

```

/// <summary>
/// 设置位置
/// </summary>
private void SetupPosition(Vector3 worldPosition)
{
    if (Camera.main == null) return;
    Vector3 screenPos = Camera.main.WorldToScreenPoint(worldPosition);
    rectTransform.position = screenPos;
}
/// <summary>
/// 播放动画序列
/// </summary>
private void PlayAnimation()
{
    isPlaying = true;
    gameObject.SetActive(true);
    // 初始状态
    rectTransform.localScale = Vector3.zero;
    canvasGroup.alpha = 1f;
}

```

```

Vector3 startPos = rectTransform.position;
// 创建动画序列
animSequence?.Kill();
animSequence = DOTween.Sequence();
// 阶段 1: 弹出 (0s -> 0.1s) - 缩放 0% → 120%
animSequence.Append(
rectTransform.DOScale(popScale, popDuration)
.SetEase(Ease.OutBack)
);
// 阶段 2: 回弹 (0.1s -> 0.2s) - 缩放 120% → 100%
animSequence.Append(
rectTransform.DOScale(1f, bounceDuration)
.SetEase(Ease.OutQuad)
);
// 阶段 3: 漂浮 (0.2s -> 0.8s) - Y 轴 +50 像素
animSequence.Append(
rectTransform.DOMoveY(startPos.y + floatDistance, floatDuration)
.SetEase(Ease.OutQuad)
);
// 阶段 4: 消失 (0.6s -> 0.8s) - 在漂浮期间淡出
// 淡出从漂浮开始后 fadeStartDelay 秒启动
animSequence.Insert(
popDuration + bounceDuration + fadeStartDelay,
canvasGroup.DOFade(0f, fadeDuration)
.SetEase(Ease.InQuad)
);
// 完成回调
animSequence.OnComplete(Complete);
}
/// <summary>
/// 动画完成, 自动销毁
/// </summary>
private void Complete()
{
isPlaying = false;
Destroy(gameObject);
}
/// <summary>
/// 设置单行组件显示/隐藏
/// </summary>
private void SetSingleRowActive(bool active)
{
{
if (iconImage != null)
{
iconImage.gameObject.SetActive(active);
}
}
if (textMesh != null)
{
textMesh.gameObject.SetActive(active);
}
}

```

```

}
/// <summary>
/// 设置双行组件显示/隐藏
/// </summary>
private void SetDualRowActive(bool active)
{
    if (costIconImage != null)
    {
        costIconImage.gameObject.SetActive(active);
    }
    if (costTextMesh != null)
    {
        costTextMesh.gameObject.SetActive(active);
    }
    if (gainIconImage != null)
    {
        gainIconImage.gameObject.SetActive(active);
    }
    if (gainTextMesh != null)
    {
        gainTextMesh.gameObject.SetActive(active);
    }
}
}
}
}
}
// =====
// FloatingTextManager.cs
// 文件位置: Assets/Scripts/UI/FloatingText/FloatingTextManager.cs
// 用途: 飘字系统管理器 (单例) - 多 Prefab 对象池 + 优先级回收
// =====
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using LightVsDecay.Core;
namespace LightVsDecay.UI.FloatingText
{
    /// <summary>
    /// 飘字系统管理器
    /// 单例模式, 管理飘字对象池和显示
    /// 支持多种 Prefab 类型 (Normal, Crit, BossShield, BossCore)
    /// </summary>
    public class FloatingTextManager : Singleton<FloatingTextManager>
    {
        //

```

```

// Inspector 配置
//

```

```

[Header("配置")]
[Tooltip("飘字配置文件")]
[SerializeField] private FloatingTextConfig config;
[Header("Canvas 引用")]
[Tooltip("飘字挂载的 Canvas（需要是 Screen Space - Overlay 或 Camera）")]
[SerializeField] private Canvas targetCanvas;
[Header("调试")]
[SerializeField] private bool showDebugInfo = false;
//

```

```

// 运行时数据
//

```

```

// 每种类型独立的对象池
private Dictionary<FloatingTextType, Queue<FloatingText>> typePools = new
Dictionary<FloatingTextType, Queue<FloatingText>>();
private List<FloatingText> activeTexts = new List<FloatingText>();
private Transform poolContainer = null;
private int totalCreated = 0;
private bool isInitialized = false;
//

```

```

// 属性
//

```

```

public int ActiveCount => activeTexts.Count;
public int TotalCreated => totalCreated;
//

```

```

// Unity 生命周期
//

```

```

protected override void OnSingletonAwake()
{
// 不在 Awake 做任何事
}
private IEnumerator Start()
{
// 等待一帧，确保所有 UI 组件都已初始化
yield return null;
if (Instance != this)
{
Debug.LogWarning("[FloatingTextManager] 非单例实例，跳过初始化");
yield break;
}
}

```

```

}
if (isInitialized)
{
yield break;
}
Initialize();
}
private void Initialize()
{
if(showDebugInfo)
Debug.Log("[FloatingTextManager] ===== 开始初始化 =====");
// 1. 验证配置
if (config == null)
{
Debug.LogError("[FloatingTextManager] 初始化失败: config 未设置! ");
return;
}
// 2. 获取 Canvas
if (targetCanvas == null)
{
targetCanvas = GetComponentInParent<Canvas>();
}
if (targetCanvas == null)
{
targetCanvas = FindObjectOfType<Canvas>();
}
if (targetCanvas == null)
{
Debug.LogError("[FloatingTextManager] 初始化失败: 找不到 Canvas! ");
return;
}
if(showDebugInfo)
Debug.Log($"[FloatingTextManager] 使用 Canvas: {targetCanvas.name}");
// 3. 创建池容器
GameObject containerGO = new GameObject("[FloatingTextPool]");
containerGO.transform.SetParent(transform, false);
RectTransform rt = containerGO.AddComponent<RectTransform>();
rt.anchorMin = Vector2.zero;
rt.anchorMax = Vector2.one;
rt.sizeDelta = Vector2.zero;
rt.anchoredPosition = Vector2.zero;
poolContainer = containerGO.transform;
if(showDebugInfo)
Debug.Log($"[FloatingTextManager] 池容器已创建: {poolContainer.name}");
// 4. 初始化各类型对象池
InitializeTypePools();
isInitialized = true;
if(showDebugInfo)
Debug.Log($"[FloatingTextManager] ===== 初始化完成: 总创建={totalCreated} =====");
}

```

```
/// <summary>
/// 初始化各类型的对象池
/// </summary>
private void InitializeTypePools()
{
    // 为每种类型创建空队列
    foreach (FloatingTextType type in System.Enum.GetValues(typeof(FloatingTextType)))
    {
        typePools[type] = new Queue<FloatingText>();
    }
    // 预热主要类型
    PrewarmType(FloatingTextType.Normal, config.prewarmCount / 2);
    PrewarmType(FloatingTextType.Crit, config.prewarmCount / 4);
    PrewarmType(FloatingTextType.BossShield, 5);
    PrewarmType(FloatingTextType.BossCore, 5);
    // 预热玩家受击飘字类型
    PrewarmType(FloatingTextType.PlayerHealthDamage, 3);
    PrewarmType(FloatingTextType.PlayerShieldDamage, 3);
    PrewarmType(FloatingTextType.PlayerHealthRestore, 2);
    PrewarmType(FloatingTextType.PlayerShieldRestore, 2);
}
/// <summary>
/// 预热指定类型的对象池
/// </summary>
private void PrewarmType(FloatingTextType type, int count)
{
    GameObject prefab = config.GetPrefab(type);
    if (prefab == null)
    {
        Debug.LogWarning($"[FloatingTextManager] {type} Prefab 未设置，跳过预热");
        return;
    }
    for (int i = 0; i < count; i++)
    {
        FloatingText ft = CreateInstance(type, prefab);
        if (ft != null)
        {
            ft.gameObject.SetActive(false);
            typePools[type].Enqueue(ft);
        }
    }
    if (showDebugInfo)
    {
        Debug.Log($"[FloatingTextManager] 预热 {type}: {count} 个");
    }
}
/// <summary>
/// 创建飘字实例
/// </summary>
private FloatingText CreateInstance(FloatingTextType type, GameObject prefab)
```

```

{
if (prefab == null || poolContainer == null) return null;
GameObject go = Instantiate(prefab, poolContainer);
go.name = $"FloatingText_{type}_{totalCreated:D3}";
FloatingText ft = go.GetComponent<FloatingText>();
if (ft == null)
{
ft = go.AddComponent<FloatingText>();
}
totalCreated++;
return ft;
}
protected override void OnSingletonDestroy()
{
ClearAll();
}
//

```

```

// 公共接口 - 伤害显示
//

```

```

/// <summary>
/// 显示普通伤害飘字（支持暴击）
/// </summary>
public void ShowDamage(Vector3 worldPosition, float damage, bool isCrit = false)
{
FloatingTextType type = isCrit ? FloatingTextType.Crit : FloatingTextType.Normal;
string text = Mathf.RoundToInt(damage).ToString();
Show(worldPosition, text, type);
}
// ═══ 【新增】碎冰伤害飘字 ═══
/// <summary>
/// 显示碎冰伤害飘字（支持暴击叠加）
/// </summary>
/// <param name="worldPosition">世界坐标</param>
/// <param name="damage">伤害值</param>
/// <param name="isCrit">是否同时触发暴击</param>
public void ShowShatterDamage(Vector3 worldPosition, float damage, bool isCrit = false)
{
// 碎冰+暴击 = ShatterCrit, 纯碎冰 = Shatter
FloatingTextType type = isCrit ? FloatingTextType.Crit : FloatingTextType.Shatter;
string text = Mathf.RoundToInt(damage).ToString();
Show(worldPosition, text, type);
}
/// <summary>
/// 显示处决飘字
/// </summary>
/// <param name="worldPosition">世界坐标</param>

```

```

public void ShowExecution(Vector3 worldPosition)
{
    Show(worldPosition, "EXECUTE!", FloatingTextType.Execution);
}
/// <summary>
/// 显示 Boss 护甲伤害飘字（银灰色 + 盾牌图标）
/// </summary>
public void ShowBossShieldDamage(Vector3 worldPosition, float damage)
{
    string text = Mathf.RoundToInt(damage).ToString();
    Show(worldPosition, text, FloatingTextType.BossShield);
}
/// <summary>
/// 显示 Boss 核心伤害飘字（红色 + 眼睛图标）
/// </summary>
/// <param name="isCrit">是否同时触发暴击（弱点+暴击叠加）</param>
public void ShowBossCoreDamage(Vector3 worldPosition, float damage, bool isCrit = false)
{
    // 如果弱点命中同时触发暴击，使用暴击样式（更大更明显）
    FloatingTextType type = isCrit ? FloatingTextType.Crit : FloatingTextType.BossCore;
    string text = Mathf.RoundToInt(damage).ToString();
    Show(worldPosition, text, type);
}
/// <summary>
/// 显示状态文本
/// </summary>
public void ShowStatus(Vector3 worldPosition, string statusText)
{
    Show(worldPosition, statusText, FloatingTextType.Status);
}
/// <summary>
/// 显示飘字（通用接口）
/// </summary>
public void Show(Vector3 worldPosition, string text, FloatingTextType type)
{
    // 如果尚未初始化，尝试立即初始化
    if (!IsInitialized)
    {
        Initialize();
    }
    if (!IsInitialized)
    {
        Debug.LogWarning("[FloatingTextManager] Show 失败: 初始化未完成");
        return;
    }
    if (showDebugInfo)
    {
        Debug.Log($"[FloatingTextManager] Show: '{text}' @ {worldPosition}, Type: {type}");
    }
    // 获取实例

```

```
FloatingText ft = GetInstance(type);
if (ft == null)
{
    Debug.LogWarning($"[FloatingTextManager] 无法获取 {type} 类型的飘字实例");
    return;
}
// 获取样式
FloatingTextStyle style = config.GetStyle(type);
int priority = config.GetPriority(type);
// 播放
ft.Play(text, worldPosition, type, style, priority, OnFloatingTextComplete);
activeTexts.Add(ft);
}
/// <summary>
/// 回收所有飘字
/// </summary>
public void ReturnAll()
{
    var list = new List<FloatingText>(activeTexts);
    foreach (var ft in list)
    {
        if (ft != null) ft.ForceStop();
    }
    activeTexts.Clear();
}
/// <summary>
/// 清空所有
/// </summary>
public void ClearAll()
{
    ReturnAll();
    foreach (var pool in typePools.Values)
    {
        while (pool.Count > 0)
        {
            var ft = pool.Dequeue();
            if (ft != null) Destroy(ft.gameObject);
        }
    }
    typePools.Clear();
    totalCreated = 0;
    isInitialized = false;
}
/// <summary>
/// 显示玩家血量受伤飘字（红色）
/// </summary>
public void ShowPlayerHealthDamage(Vector3 worldPosition, int damage)
{
    // 调整 Y 坐标：塔在-10，飘字在-8
    Vector3 adjustedPos = new Vector3(worldPosition.x, -8f, worldPosition.z);
```

```

string text = $"-{damage}";
Show(adjustedPos, text, FloatingTextType.PlayerHealthDamage);
}
/// <summary>
/// 显示玩家护盾受伤飘字（青色）
/// </summary>
public void ShowPlayerShieldDamage(Vector3 worldPosition, int damage)
{
    // 调整 Y 坐标：塔在-10，飘字在-8
    Vector3 adjustedPos = new Vector3(worldPosition.x, -8f, worldPosition.z);
    string text = $"-{damage}";
    Show(adjustedPos, text, FloatingTextType.PlayerShieldDamage);
}
/// <summary>
/// 显示玩家血量恢复飘字（绿色）
/// </summary>
public void ShowPlayerHealthRestore(Vector3 worldPosition, int amount)
{
    Vector3 adjustedPos = new Vector3(worldPosition.x, -8f, worldPosition.z);
    string text = $"#{amount}";
    Show(adjustedPos, text, FloatingTextType.PlayerHealthRestore);
}
/// <summary>
/// 显示玩家护盾恢复飘字（青色+）
/// </summary>
public void ShowPlayerShieldRestore(Vector3 worldPosition, int amount)
{
    Vector3 adjustedPos = new Vector3(worldPosition.x, -8f, worldPosition.z);
    string text = $"#{amount}";
    Show(adjustedPos, text, FloatingTextType.PlayerShieldRestore);
}
//

```

```

// 私有方法
//

```

```

private FloatingText GetInstance(FloatingTextType requestType)
{
    // 确保类型池存在
    if (!typePools.ContainsKey(requestType))
    {
        typePools[requestType] = new Queue<FloatingText>();
    }
    var pool = typePools[requestType];
    // 1. 从对应类型池中取
    if (pool.Count > 0)
    {
        return pool.Dequeue();
    }
}

```

```
}  
// 2. 动态创建  
if (totalCreated < config.maxPoolSize && poolContainer != null)  
{  
    GameObject prefab = config.GetPrefab(requestType);  
    if (prefab != null)  
    {  
        return CreateInstance(requestType, prefab);  
    }  
}  
// 3. 优先级回收  
return TryRecycleLowPriority(requestType);  
}  
private FloatingText TryRecycleLowPriority(FloatingTextType requestType)  
{  
    int requestPriority = config.GetPriority(requestType);  
    FloatingText candidate = null;  
    float minScore = float.MaxValue;  
    foreach (var ft in activeTexts)  
    {  
        if (ft == null || !ft.IsPlaying) continue;  
        if (ft.Priority > requestPriority) continue;  
        float score = ft.Priority * 100f + ft.RemainingPercent * 100f;  
        if (score < minScore)  
        {  
            minScore = score;  
            candidate = ft;  
        }  
    }  
    if (candidate != null)  
    {  
        activeTexts.Remove(candidate);  
        candidate.Reset();  
        return candidate;  
    }  
    return null;  
}  
private void OnFloatingTextComplete(FloatingText ft)  
{  
    if (ft == null) return;  
    activeTexts.Remove(ft);  
    ft.Reset();  
    // 根据类型放回对应的池  
    FloatingTextType type = ft.CurrentType;  
    if (!typePools.ContainsKey(type))  
    {  
        typePools[type] = new Queue<FloatingText>();  
    }  
    typePools[type].Enqueue(ft);  
}
```

```
//  
  
// 调试 GUI  
//  
  
#if UNITY_EDITOR  
private void OnGUI()  
{  
    if (!showDebugInfo || !Application.isPlaying) return;  
    GUILayout.BeginArea(new Rect(10, 450, 250, 180));  
    GUILayout.Label("== FloatingText Debug ==");  
    GUILayout.Label($"Initialized: {isInitialized}");  
    GUILayout.Label($"Active: {activeTexts.Count}");  
    GUILayout.Label($"Total Created: {totalCreated}");  
    // 显示各类型池的数量  
    foreach (var kvp in typePools)  
    {  
        GUILayout.Label($"  {kvp.Key}: {kvp.Value.Count}");  
    }  
    GUILayout.EndArea();  
}  
#endif  
}  
}  
// =====  
// FloatingTextConfig.cs  
// 文件位置: Assets/Scripts/UI/FloatingText/FloatingTextConfig.cs  
// 用途: 飘字视觉配置 (ScriptableObject)  
// =====  
using UnityEngine;  
namespace LightVsDecay.UI.FloatingText  
{  
    /// <summary>  
    /// 单个飘字类型的配置  
    /// </summary>  
    [System.Serializable]  
    public class FloatingTextStyle  
    {  
        [Header("颜色")]  
        [Tooltip("文字颜色")]  
        public Color textColor = Color.white;  
        [Tooltip("描边颜色")]  
        public Color outlineColor = Color.black;  
        [Header("字体")]  
        [Tooltip("字体大小")]  
        [Range(16f, 72f)]  
        public float fontSize = 32f;  
        [Tooltip("是否加粗")]
```

```

public bool isBold = false;
[Tooltip("描边宽度")]
[Range(0f, 0.5f)]
public float outlineWidth = 0.2f;
[Header("动画")]
[Tooltip("持续时间")]
[Range(0.3f, 2f)]
public float duration = 0.6f;
[Tooltip("初始向上速度")]
[Range(0f, 300f)]
public float initialUpSpeed = 150f;
[Tooltip("水平随机范围")]
[Range(0f, 200f)]
public float horizontalRandomRange = 80f;
[Tooltip("重力（下落加速度）")]
[Range(0f, 500f)]
public float gravity = 0f;
[Tooltip("淡出开始时间（占总时长百分比）")]
[Range(0.3f, 0.9f)]
public float fadeStartPercent = 0.5f;
[Header("缩放动画")]
[Tooltip("是否启用缩放动画")]
public bool useScaleAnimation = false;
[Tooltip("初始缩放")]
[Range(0.5f, 2f)]
public float initialScale = 1f;
[Tooltip("峰值缩放")]
[Range(1f, 3f)]
public float peakScale = 1.5f;
[Tooltip("缩放峰值时间（占总时长百分比）")]
[Range(0.1f, 0.5f)]
public float scalePeakPercent = 0.2f;
[Header("整体缩放倍率")]
[Tooltip("整体大小倍率（影响字体和图标）")]
[Range(0.5f, 2f)]
public float sizeMultiplier = 1f;
}
/// <summary>
/// 飘字系统配置（ScriptableObject）
/// </summary>
[CreateAssetMenu(fileName = "FloatingTextConfig", menuName =
"LightVsDecay/FloatingTextConfig")]
public class FloatingTextConfig : ScriptableObject
{
//

```

```

// 对象池设置
//

```

```
[Header("对象池设置")]
[Tooltip("预热数量")]
[Range(10, 50)]
public int prewarmCount = 20;
[Tooltip("最大数量上限")]
[Range(20, 100)]
public int maxPoolSize = 40;
//
```

```
// Prefab 引用
//
```

```
[Header("Prefab 引用")]
[Tooltip("普通伤害 Prefab")]
public GameObject normalPrefab;
[Tooltip("暴击伤害 Prefab")]
public GameObject critPrefab;
[Tooltip("Boss 护甲伤害 Prefab")]
public GameObject bossShieldPrefab;
[Tooltip("Boss 核心伤害 Prefab")]
public GameObject bossCorePrefab;
[Tooltip("状态文本 Prefab（可选，不设置则使用 Normal）")]
public GameObject statusPrefab;
[Header("玩家受击飘字 Prefab")]
[Tooltip("玩家血量受伤 Prefab（可选，不设置则使用 Normal）")]
public GameObject playerHealthDamagePrefab;
[Tooltip("玩家护盾受伤 Prefab（可选，不设置则使用 Normal）")]
public GameObject playerShieldDamagePrefab;
[Tooltip("玩家恢复飘字 Prefab（可选，不设置则使用 Normal）")]
public GameObject playerRestorePrefab;
//
```

```
// 优先级设置
//
```

```
[Header("优先级设置")]
[Tooltip("普通伤害优先级（越低越容易被回收）")]
public int normalPriority = 0;
[Tooltip("暴击伤害优先级")]
public int critPriority = 3;
[Tooltip("状态文本优先级")]
public int statusPriority = 1;
[Tooltip("Boss 护甲伤害优先级")]
public int bossShieldPriority = 1;
[Tooltip("Boss 核心伤害优先级")]
```

```

public int bossCorePriority = 2;
[Header("玩家受击飘字优先级")]
[Tooltip("玩家血量受伤优先级")]
public int playerHealthDamagePriority = 4; // 高优先级
[Tooltip("玩家护盾受伤优先级")]
public int playerShieldDamagePriority = 3;
[Tooltip("玩家恢复优先级")]
public int playerRestorePriority = 2;
[Header("碎冰优先级")]
[Range(0, 100)]
public int shatterPriority = 55;
[Range(0, 100)]
public int shatterCritPriority = 85;
[Range(0, 100)]
public int executionPriority = 90;
//

```

```

// 样式配置
//

```

```

[Header("普通伤害样式")]
public FloatingTextStyle normalStyle = new FloatingTextStyle
{
    textColor = Color.white,
    outlineColor = new Color(0.2f, 0.2f, 0.2f, 1f),
    fontSize = 32f,
    isBold = false,
    outlineWidth = 0.15f,
    duration = 0.6f,
    initialUpSpeed = 120f,
    horizontalRandomRange = 60f,
    gravity = 100f,
    fadeStartPercent = 0.5f,
    useScaleAnimation = false,
    initialScale = 1f,
    peakScale = 1f,
    scalePeakPercent = 0.2f,
    sizeMultiplier = 1.0f // 标准大小
};
[Header("暴击伤害样式")]
public FloatingTextStyle critStyle = new FloatingTextStyle
{
    textColor = new Color(1f, 0f, 0.33f, 1f), // #FF0055 霓虹红
    outlineColor = new Color(0.5f, 0f, 0.15f, 1f),
    fontSize = 48f,
    isBold = true,
    outlineWidth = 0.25f,
    duration = 1.0f,

```

```
initialUpSpeed = 200f,
horizontalRandomRange = 40f,
gravity = 300f,
fadeStartPercent = 0.6f,
useScaleAnimation = true,
initialScale = 0.8f,
peakScale = 1.4f,
scalePeakPercent = 0.15f,
sizeMultiplier = 1.5f // 1.5 倍大小
};
[Header("状态文本样式")]
public FloatingTextStyle statusStyle = new FloatingTextStyle
{
    textColor = new Color(1f, 0.92f, 0.016f, 1f), // 黄色
    outlineColor = new Color(0.3f, 0.25f, 0f, 1f),
    fontSize = 36f,
    isBold = true,
    outlineWidth = 0.2f,
    duration = 0.8f,
    initialUpSpeed = 180f,
    horizontalRandomRange = 20f,
    gravity = 50f,
    fadeStartPercent = 0.6f,
    useScaleAnimation = true,
    initialScale = 0.6f,
    peakScale = 1.2f,
    scalePeakPercent = 0.25f,
    sizeMultiplier = 1.0f
};
[Header("Boss 护甲伤害样式")]
public FloatingTextStyle bossShieldStyle = new FloatingTextStyle
{
    textColor = new Color(0.8f, 0.8f, 0.8f, 1f), // 银灰色 #CCCCCC
    outlineColor = new Color(0.3f, 0.3f, 0.3f, 1f),
    fontSize = 22f, // 比普通小 30%
    isBold = false,
    outlineWidth = 0.1f,
    duration = 0.5f,
    initialUpSpeed = 100f,
    horizontalRandomRange = 40f,
    gravity = 80f,
    fadeStartPercent = 0.4f,
    useScaleAnimation = false,
    initialScale = 1f,
    peakScale = 1f,
    scalePeakPercent = 0.2f,
    sizeMultiplier = 0.7f // 0.7 倍大小
};
[Header("Boss 核心伤害样式")]
public FloatingTextStyle bossCoreStyle = new FloatingTextStyle
```

```
{
textColor = new Color(1f, 0.2f, 0.1f, 1f), // 深红色
outlineColor = new Color(0.6f, 0f, 0f, 1f), // 红色描边
fontSize = 42f,
isBold = true,
outlineWidth = 0.3f,
duration = 0.9f,
initialUpSpeed = 180f,
horizontalRandomRange = 30f,
gravity = 200f,
fadeStartPercent = 0.55f,
useScaleAnimation = true,
initialScale = 0.7f,
peakScale = 1.3f,
scalePeakPercent = 0.2f,
sizeMultiplier = 1.3f // 1.3 倍大小
};
[Header("玩家血量受伤样式")]
public FloatingTextStyle playerHealthDamageStyle = new FloatingTextStyle
{
textColor = new Color(1f, 0.2f, 0.2f, 1f), // 红色 #FF3333
outlineColor = new Color(0.3f, 0f, 0f, 1f), // 深红描边
fontSize = 38f,
isBold = true,
outlineWidth = 0.25f,
duration = 0.8f,
initialUpSpeed = 160f,
horizontalRandomRange = 30f,
gravity = 150f,
fadeStartPercent = 0.5f,
useScaleAnimation = true,
initialScale = 0.8f,
peakScale = 1.3f,
scalePeakPercent = 0.2f,
sizeMultiplier = 1.2f
};
[Header("玩家护盾受伤样式")]
public FloatingTextStyle playerShieldDamageStyle = new FloatingTextStyle
{
textColor = new Color(0f, 0.9f, 1f, 1f), // 青色 #00E5FF
outlineColor = new Color(0f, 0.3f, 0.4f, 1f), // 深青描边
fontSize = 36f,
isBold = true,
outlineWidth = 0.2f,
duration = 0.7f,
initialUpSpeed = 150f,
horizontalRandomRange = 35f,
gravity = 120f,
fadeStartPercent = 0.5f,
useScaleAnimation = true,
```



```
initialScale = 0.8f,
peakScale = 1.2f,
scalePeakPercent = 0.2f,
sizeMultiplier = 1.1f
};
[Header("玩家血量恢复样式")]
public FloatingTextStyle playerHealthRestoreStyle = new FloatingTextStyle
{
    textColor = new Color(0.2f, 1f, 0.4f, 1f),    // 绿色 #33FF66
    outlineColor = new Color(0f, 0.3f, 0.1f, 1f), // 深绿描边
    fontSize = 34f,
    isBold = true,
    outlineWidth = 0.2f,
    duration = 0.9f,
    initialUpSpeed = 140f,
    horizontalRandomRange = 20f,
    gravity = 100f,
    fadeStartPercent = 0.6f,
    useScaleAnimation = true,
    initialScale = 0.9f,
    peakScale = 1.2f,
    scalePeakPercent = 0.25f,
    sizeMultiplier = 1.0f
};
[Header("玩家护盾恢复样式")]
public FloatingTextStyle playerShieldRestoreStyle = new FloatingTextStyle
{
    textColor = new Color(0.4f, 1f, 1f, 1f),      // 亮青色 #66FFFF
    outlineColor = new Color(0f, 0.4f, 0.4f, 1f), // 深青描边
    fontSize = 34f,
    isBold = true,
    outlineWidth = 0.2f,
    duration = 0.9f,
    initialUpSpeed = 140f,
    horizontalRandomRange = 20f,
    gravity = 100f,
    fadeStartPercent = 0.6f,
    useScaleAnimation = true,
    initialScale = 0.9f,
    peakScale = 1.2f,
    scalePeakPercent = 0.25f,
    sizeMultiplier = 1.0f
};
[Header("碎冰伤害样式")]
public FloatingTextStyle shatterStyle = new FloatingTextStyle
{
    textColor = new Color(0f, 0.75f, 1f, 1f),    // 亮蓝色 #00BFFF
    outlineColor = new Color(0f, 0.2f, 0.4f, 1f), // 深蓝描边
    fontSize = 36f,
    isBold = true,
```

```
outlineWidth = 0.2f,
duration = 0.7f,
initialUpSpeed = 160f,
horizontalRandomRange = 50f,
gravity = 120f,
fadeStartPercent = 0.5f,
useScaleAnimation = true,
initialScale = 0.8f,
peakScale = 1.2f,
scalePeakPercent = 0.2f,
sizeMultiplier = 1.15f // 比普通大 15%
};
[Header("碎冰暴击样式")]
public FloatingTextStyle shatterCritStyle = new FloatingTextStyle
{
    textColor = new Color(0.4f, 0.9f, 1f, 1f), // 冰蓝色 #66E5FF
    outlineColor = new Color(0.8f, 0.1f, 0.1f, 1f), // 红色描边 (高光时刻)
    fontSize = 44f,
    isBold = true,
    outlineWidth = 0.3f,
    duration = 0.9f,
    initialUpSpeed = 200f,
    horizontalRandomRange = 40f,
    gravity = 180f,
    fadeStartPercent = 0.55f,
    useScaleAnimation = true,
    initialScale = 0.6f,
    peakScale = 1.5f,
    scalePeakPercent = 0.15f,
    sizeMultiplier = 1.4f // 比普通大 40%
};
[Header("处决样式")]
public FloatingTextStyle executionStyle = new FloatingTextStyle
{
    textColor = new Color(0.2f, 0.8f, 1f, 1f), // 冰蓝色
    outlineColor = new Color(0.6f, 0f, 0.8f, 1f), // 紫色描边 (特殊感)
    fontSize = 40f,
    isBold = true,
    outlineWidth = 0.35f,
    duration = 1.0f,
    initialUpSpeed = 180f,
    horizontalRandomRange = 20f,
    gravity = 100f,
    fadeStartPercent = 0.6f,
    useScaleAnimation = true,
    initialScale = 0.5f,
    peakScale = 1.6f,
    scalePeakPercent = 0.2f,
    sizeMultiplier = 1.3f
};
```